

BASES TECNOLÓGICAS PARA EL SERVICIO

MÓDULO: PHP

ACTIVIDAD: Avance de la práctica –
Segundo avance. Práctica Desarrollo de
APIs REST

ALUMNO: CESAR CEDILLO
RODRÍGUEZ

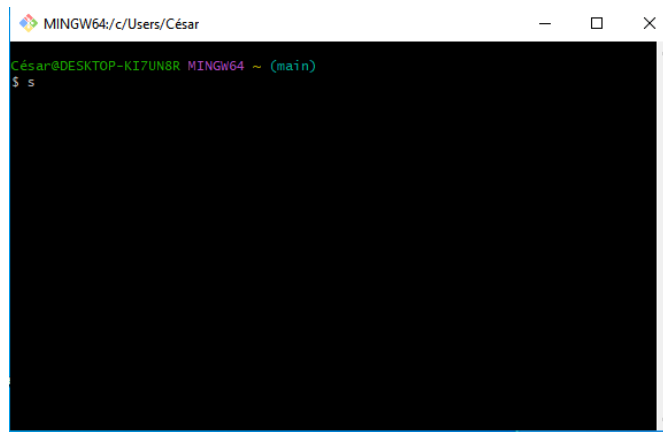
GRUPO: 8

DOCENTE: JUAN MANUEL
TOLENTINO MORALES

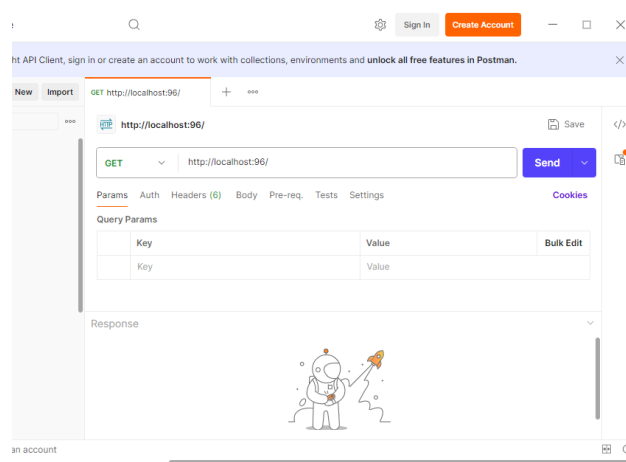
REPORTE DE AVANCES DE LA PRÁCTICA

En este reporte se describe el avance de la práctica de **Desarrollo de APIs REST con Laravel**. Para el desarrollo del proyecto se está utilizando **Docker**, con el fin de facilitar la configuración del entorno y asegurar que sea portable.. Actualmente se ha completado la fase de preparación del entorno de trabajo.

Como primer paso, se verificó que todas las herramientas necesarias estuvieran instaladas correctamente. Entre ellas se encuentran **Git** para el control de versiones, **Docker Desktop** para la gestión de contenedores, **Visual Studio Code** como editor de código y **Postman** para realizar pruebas de la API. Al contar con todas estas herramientas, fue posible avanzar sin mayores inconvenientes.



Consola git



postman

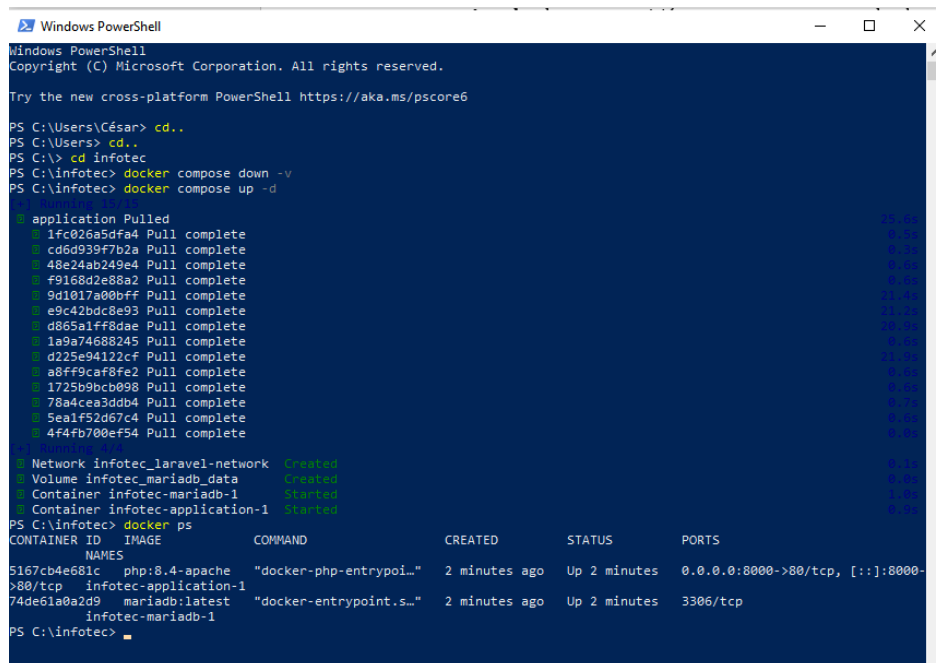
Posteriormente, se creó la estructura inicial del proyecto. Se generó una carpeta principal llamada **infotec**, dentro de la cual se creó la carpeta **src**, destinada a contener todo el código de la aplicación Laravel. Esta estructura es sencilla, pero permite mantener el proyecto ordenado desde el inicio y seguir buenas prácticas de desarrollo.

Una vez creada la estructura, se abrió el proyecto en **Visual Studio Code** directamente desde la carpeta *infotec*. Esto facilitó el acceso a los archivos y permitió trabajar de forma más cómoda, ya que el editor muestra toda la estructura del proyecto sin necesidad de estar navegando constantemente entre carpetas desde la terminal.

El paso más importante en esta etapa fue la creación y configuración del archivo **docker-compose.yml**, el cual define los servicios necesarios para el funcionamiento del proyecto. Inicialmente, se intentó utilizar la imagen de **Bitnami para Laravel**, sin embargo, al levantar los contenedores se presentó el error `chown: invalid group: 'bitnami:bitnami'`, relacionado con problemas de permisos dentro del contenedor. Este error impedía que la aplicación se ejecutara correctamente.

Debido a este inconveniente, fue necesario **modificar el archivo YAML** y cambiar la imagen utilizada para la aplicación. Como solución, se optó por utilizar una **imagen estándar de PHP 8.4 con Apache**, lo que permitió tener un mayor control sobre el entorno y evitar los problemas de permisos. A partir de este cambio, la instalación de Laravel y sus dependencias se realizó directamente dentro del contenedor.

De este modo, la fase de preparación del entorno se completó de manera satisfactoria. A pesar del problema inicial con la imagen de Bitnami, se logró una solución funcional y estable utilizando una imagen estándar de PHP. El proyecto cuenta con una estructura clara, una base de datos configurada y un entorno listo para comenzar con el desarrollo de la API.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\César> cd..
PS C:\Users> cd..
PS C:\> cd infotec
PS C:\infotec> docker compose down -v
PS C:\infotec> docker compose up -d

# application Pulled
# 1fc026a5dfa4 Pull complete
# cd6d939f7b2a Pull complete
# 48e24ab249e4 Pull complete
# f9168de288e2 Pull complete
# 9d1017a00b9f Pull complete
# e9c42bdc8e93 Pull complete
# d865a1ff8dae Pull complete
# 1a9a74688245 Pull complete
# d225e94122cf Pull complete
# a8ff9caf8fe2 Pull complete
# 1725b9bcb098 Pull complete
# 78a4cea3ddb4 Pull complete
# 5ealf52d67c4 Pull complete
# 4f4fb700ef54 Pull complete

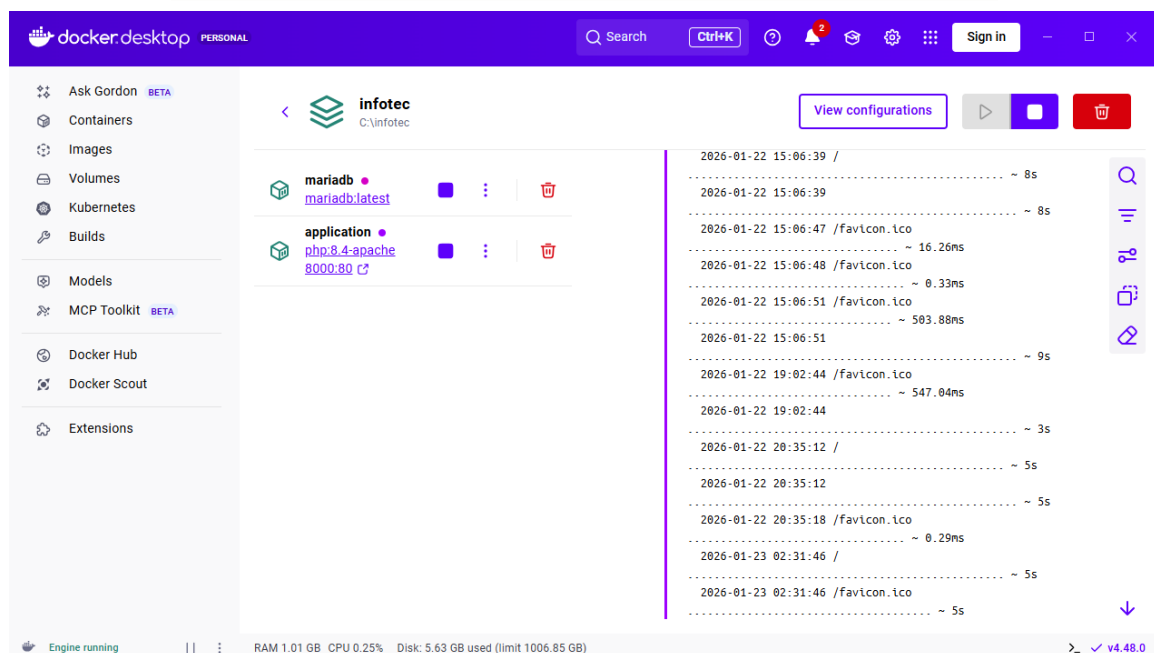
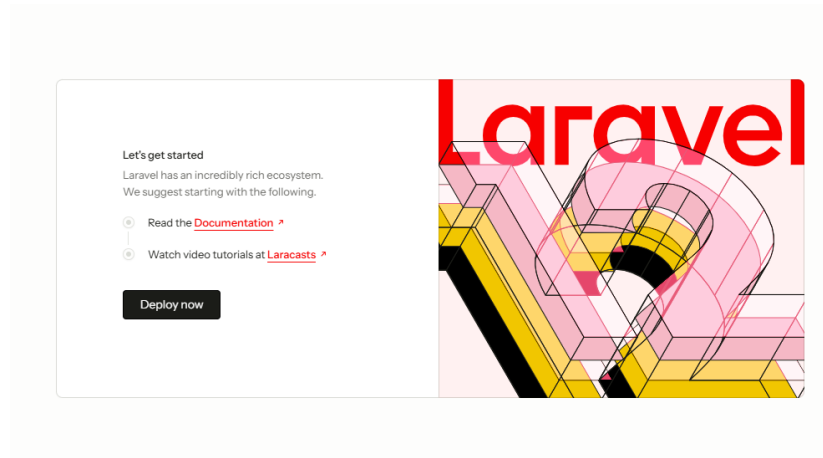
# Network infotec_laravel-network Created
# Volume infotec_mariadb_data Created
# Container infotec-mariadb-1 Started
# Container infotec-application-1 Started

PS C:\infotec> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
5167cb4e681c   php:8.4-apache                    "docker-php-entrypoi..." 2 minutes ago  Up 2 minutes  0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
74de61a0a2d9   infotec-application-1            "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  3306/tcp
74de61a0a2d9   infotec-mariadb-1                "docker-entrypoint.s..." 2 minutes ago  Up 2 minutes  3306/tcp
```

Como siguientes pasos, se planea levantar los contenedores, verificar que la aplicación funcione correctamente y comenzar con el desarrollo de la API en Laravel. Esto incluirá la creación de

modelos, controladores, rutas, migraciones de base de datos y pruebas para asegurar que las operaciones básicas funcionen correctamente.

Por el momento, el proyecto avanza de forma adecuada y este reporte se actualizará conforme continúe la práctica.



ETAPA 2

En esta etapa se tiene como objetivo diseñar e implementar la estructura base de una API REST en Laravel para la gestión de eventos, utilizando controladores API, modelos, migraciones y rutas, preparando el proyecto para su posterior prueba mediante Postman.

Verificación del uso de Artisan

Se verificó que el entorno Laravel permite la ejecución de comandos Artisan desde el contenedor Docker, lo cual confirma que el framework se encuentra correctamente instalado y funcional.

```
PS C:\infotec> docker compose exec application php artisan --version
Laravel Framework 12.48.1
PS C:\infotec>
```

2.2 Configuración de la base de datos

Se configuró Laravel para utilizar el servicio de base de datos MariaDB definido en Docker, actualizando las variables de conexión en el archivo .env y limpiando la caché de configuración para aplicar los cambios.

```
DB_CONNECTION=mysql
DB_HOST=mariadb
DB_PORT=3306
DB_DATABASE=bitnami_myapp
DB_USERNAME=bn_myapp
DB_PASSWORD=bitnam1_my@pp
```

2.3 Creación del modelo, controlador y migración

Se generó el modelo Evento junto con su controlador API y la migración correspondiente mediante el uso de Artisan, lo que permitió establecer la base de la entidad principal del sistema.

2.5 Configuración del modelo Evento

Se configuró el modelo Evento para permitir la asignación masiva de los campos definidos en la migración y se especificó la tabla asociada en la base de datos.

```
1  <?php
2  namespace App\Models;
3  use Illuminate\Database\Eloquent\Model;
4  class Evento extends Model
5  {
6      protected $table = 'eventos';
7      protected $fillable = [
8          'titulo',
9          'descripcion',
10         'fecha_inicio',
11         'fecha_fin',
12         'ubicacion'
13     ];
14 }
```

2.7 Configuración de rutas de la API

Se creó y configuró el archivo routes/api.php, donde se definieron las rutas REST que exponen los endpoints para la gestión de eventos mediante el prefijo /api.

2.8 Verificación de las rutas

Se verificó que las rutas de la API se encuentren correctamente registradas en Laravel mediante el listado de rutas.

```
C:\infotec>
```

```
GET|HEAD /
.....
.....
.....

GET|HEAD storage/{path}
.....
.....
..... storage.local

GET|HEAD up ....
```

```
PS C:\infotec> docker compose exec application php artisan --version
Laravel Framework 12.48.1
PS C:\infotec>
PS C:\infotec> docker compose exec application ls database/migration
ls: cannot access 'database/migration': No such file or directory
PS C:\infotec> docker compose exec application pwd
/var/www/html
PS C:\infotec> docker compose exec application ls
README.md  artisan  composer.json  config  licenses  phpunit.xml  resources  storage  vendor
app        bootstrap  composer.lock  database  package.json  public  routes  tests  vite.config.js
PS C:\infotec> docker compose exec application ls database/migrations
0001_01_01_000000_create_users_table.php  0001_01_01_000002_create_jobs_table.php
0001_01_01_000001_create_cache_table.php  2026_01_29_193948_create_eventos_table.php
PS C:\infotec> docker compose exec application php artisan migrate

INFO: Preparing database.

Creating migration table ..... 477.54ms DONE

INFO: Running migrations.

0001_01_01_000000_create_users_table ..... 693.36ms DONE
0001_01_01_000001_create_cache_table ..... 34.99ms DONE
0001_01_01_000002_create_jobs_table ..... 369.17ms DONE
2026_01_29_193948_create_eventos_table ..... 17.89ms DONE

PS C:\infotec> docker compose exec application ls routes
console.php  web.php
PS C:\infotec> docker compose exec application touch routes/api.php
PS C:\infotec> docker compose exec application ls routes
api.php  console.php  web.php
```

RESUMEN DE LA ETAPA 2

En esta etapa se desarrolló la estructura base de una API REST utilizando Laravel, orientada a la gestión de eventos. Se crearon el modelo, la migración y el controlador API correspondientes a la entidad *Evento*, definiendo la estructura de la base de datos y la lógica básica para las operaciones CRUD. Asimismo, se configuraron las rutas de la API en el archivo `api.php`, quedando registrados los endpoints necesarios para el acceso a los recursos mediante solicitudes HTTP y dejando la aplicación preparada para su validación en la siguiente etapa.